

Introduction to Python

Python - a mysterious name

Python is a widely used general-purpose, high level programming language. It was initially designed by Dutch programmer **Guido van Rossum in 1991.**

The name Python comes from an old BBC television comedy sketch series called Monty Python's Flying Circus. When Guido van Rossum was creating Python, he was also reading the scripts of Monty Python. He thought the name Python was appropriately short and slightly mysterious.



Why should we learn Python?

- Python is a higher level programming language.
- Its syntax allows programmers to express concepts in fewer lines of code.
- Python is a programming language that lets you work quickly and integrate systems more efficiently.
- One can plot figures using Python.
- One can perform symbolic mathematics easily using Python.
- It is available freely online.

Python versions

Python was first released on February 20, 1991 and later on developed by Python Software Foundation.

Major Python versions are – **Python 1, Python 2 and Python 3.**

- On 26th January 1994, **Python 1.0** was released.
- On 16th October 2000, **Python 2.0** was released with many new features.
- On 3rd December 2008, **Python 3.0** was released with more testing and includes new features.

Latest version - On 2nd October 2023, **Python 3.12** was released.

To check your Python version

- i) For Linux OS type **python -V** in the terminal window.
- ii) For Windows and MacOS type `import sys print(sys.version)` in the interactive shell.

Python Interpreter

The program that translates Python instructions and then executes them is the Python interpreter. When we write a Python program, the program is executed by the Python interpreter. This interpreter is written in the C language.

There are certain online interpreters like

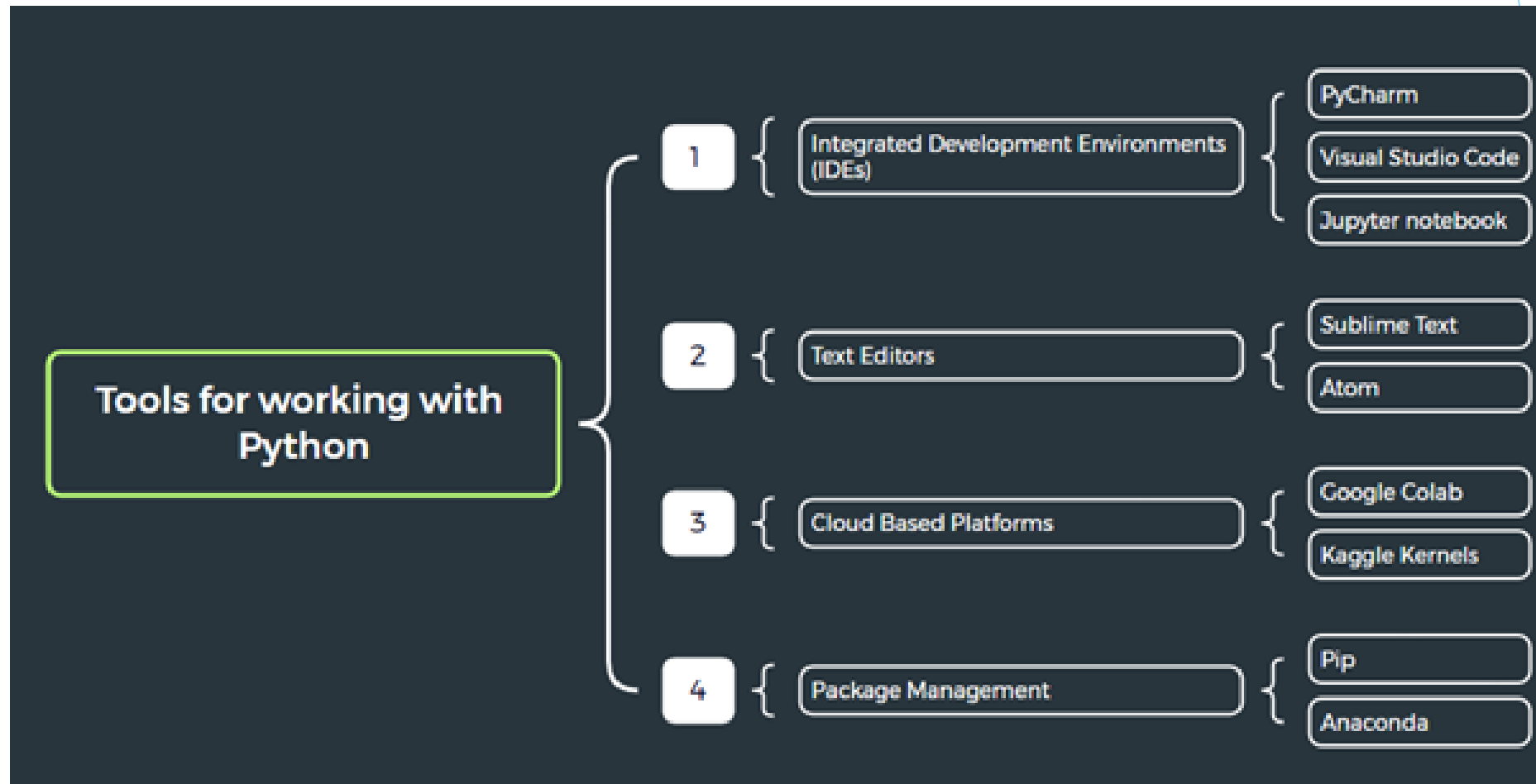
<https://ide.geeksforgeeks.org/>,

<http://ideone.com/> ,

<http://codepad.org/>

that can be used to start Python without installing an interpreter.

Libraries and Environments



Libraries and Environments

Integrated Development Environments (IDEs)

PyCharm is a robust IDE with features like code analysis, debugging tools, and integration with version control systems. It is particularly well-suited for larger projects.

Visual Studio Code is a lightweight and highly customizable code editor that supports Python through extensions, offering features like IntelliSense, debugging, and Git integration.

Jupyter Notebook is an open-source web application that allows users to create and share documents containing code, equations, visualizations, and narrative text. Jupyter is especially popular in data science and academia.

Libraries and Environments

Text Editors

Sublime Text is a fast and responsive text editor that supports multiple programming languages, including Python. It offers features like syntax highlighting, code snippets, and customizable key bindings.

Cloud-Based Platforms

Google Colab is a cloud-based Jupyter Notebook environment that allows you to write and execute Python code directly in your browser. It is especially popular for machine learning tasks due to its free access to GPUs and TPUs.

Kaggle Kernels is a cloud-based environment that supports Jupyter notebooks and provides access to datasets and machine learning resources, ideal for data analysis competitions.

Package Management

PIP is a package installer for Python, allowing you to install and manage additional libraries and dependencies easily.

Anaconda is a popular distribution that simplifies package management and deployment for scientific computing and data science, including a package manager called Conda.

Python IDLE

Python interpreter is embedded in a number of larger programs that make it particularly easy to develop Python programs. Such a programming environment is IDLE

(**Integrated Development and Learning Environment**).

It is available freely online. For Windows machine IDLE (Integrated Development and Learning Environment) is installed when you install Python.

Running Python

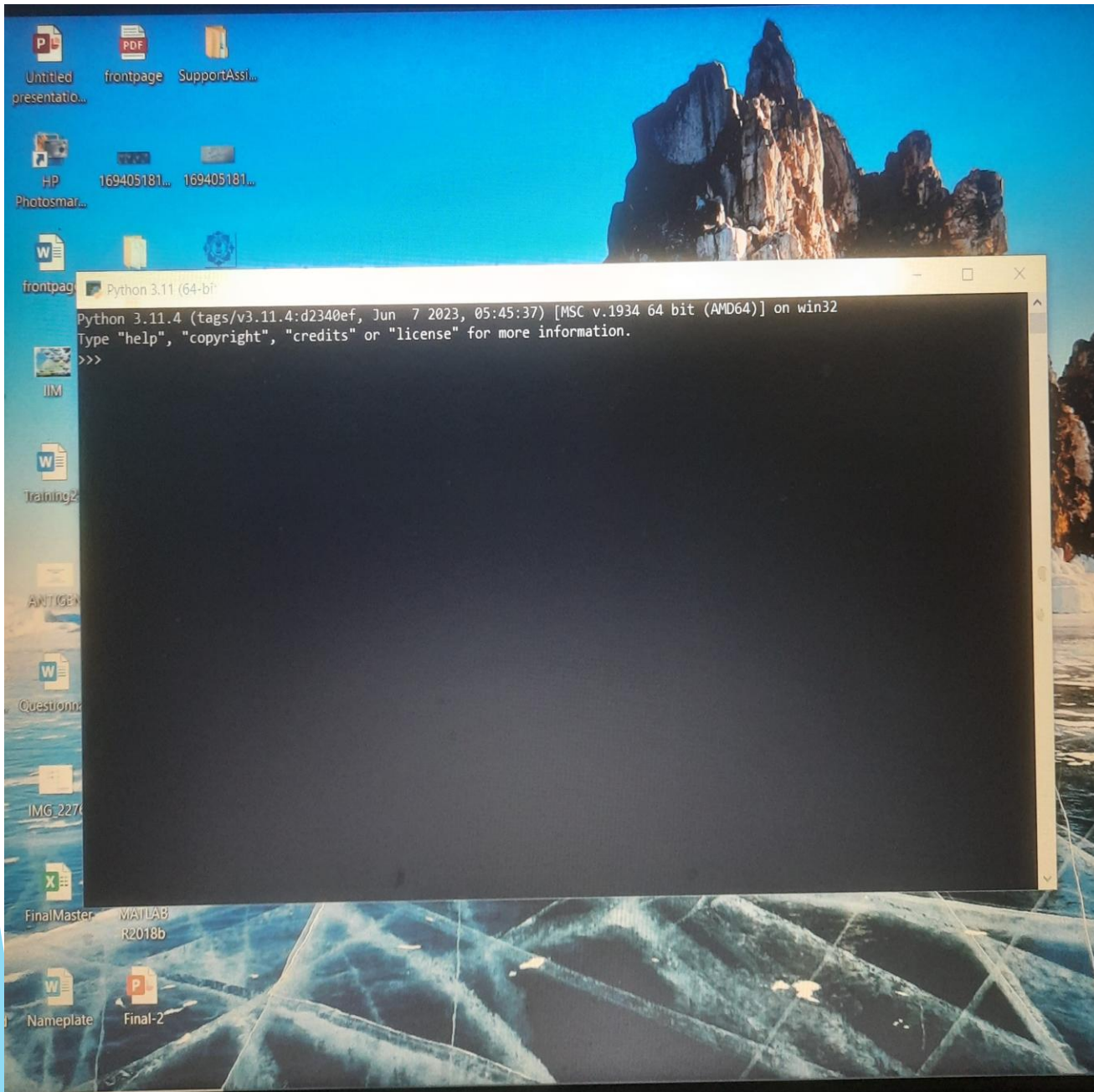
There are two modes for using the Python interpreter:

1) Interactive Mode

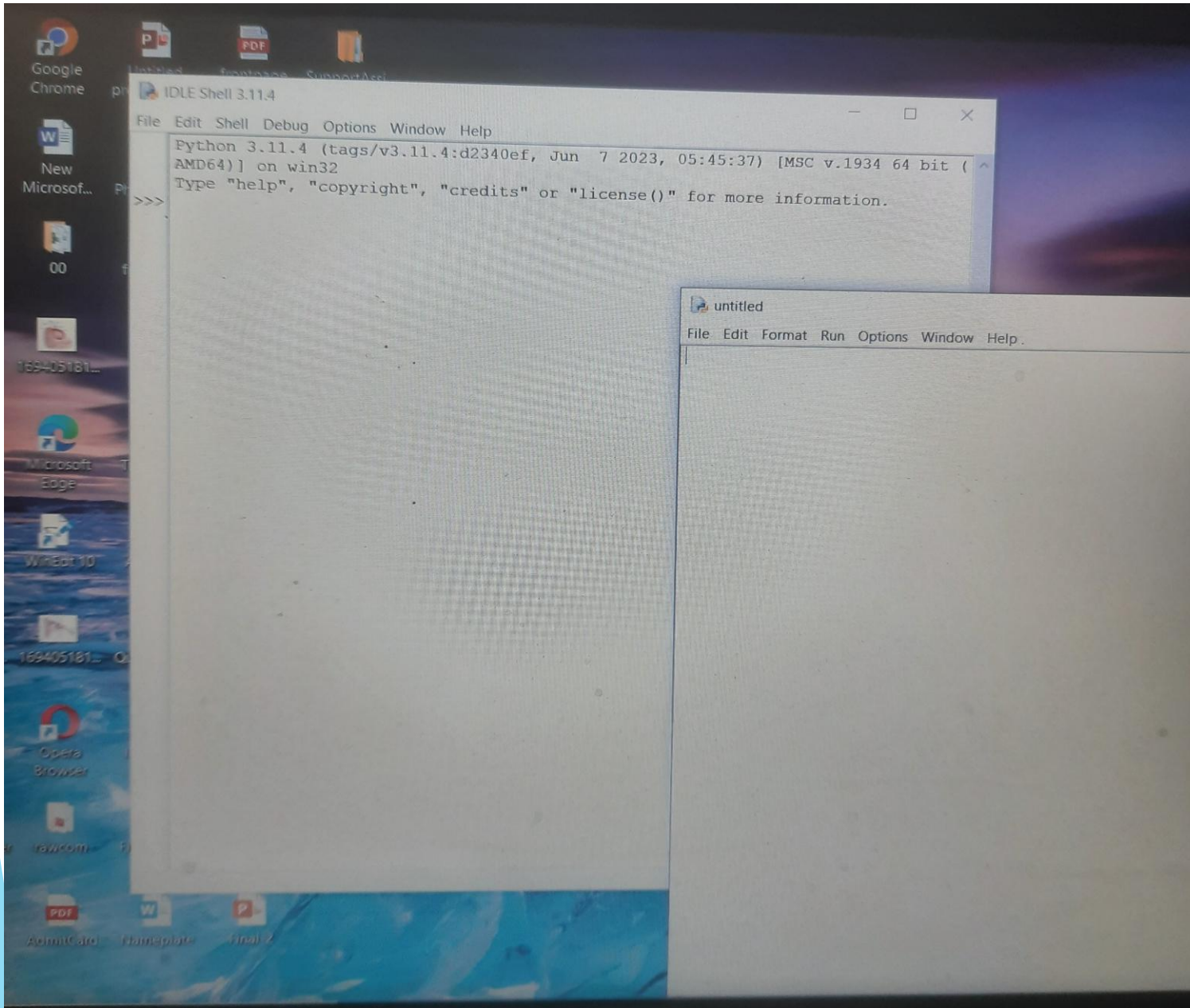
2) Script Mode

Options for running the program:

- In Windows, you can display your folder contents, and double click on madlib.py to start the program.
- In Linux or on a Mac you can open a terminal window, change into your python directory, and enter the command `python madlib.py`



Interactive shell



IDLE shell

Running Python

1) in interactive mode:

```
>>> print("Hello Teachers")
```

```
Hello Teachers
```

```
>>> a=10
```

```
>>> print(a)
```

```
10
```

```
>>> x=10
```

```
>>> z=x+20
```

```
>>> z
```

```
30
```

Running Python

2) in script mode:

Programmers can store Python script source code in a file with the .py extension, and use the interpreter to execute the contents of the file.

For UNIX OS to run a script file MyFile.py you have to type:

python MyFile.py

Data Types

Python has various standard data types:

- Integer [class 'int']
- Float [class 'float']
- Boolean [class 'bool']
- String [class 'str']

Integer

Int:

For integer or whole number, positive or negative, without decimals of unlimited length.

```
>>> print(2465635468765)
```

```
2465635468765
```

```
>>> print(0b10)      # 0b indicates binary number
```

```
2
```

```
>>> print(0x10)      # 0x indicates hexadecimal number
```

```
16
```

```
>>> a=11
```

```
>>> print(type(a))
```

```
<class 'int'>
```


Float

Float:

Float, or "floating point number" is a number, positive or negative.

Float can also be scientific numbers with an "e" to indicate the power of 10.

```
>>> y=2.8
```

```
>>> y
```

```
2.8
```

```
>>> print(0.00000045)
```

```
4.5e-07
```

```
>>> y=2.8
```

```
>>> print(type(y))
```

```
<class 'float'>
```

Boolean and String

Boolean:

Objects of Boolean type may have one of two values, True or False:

```
>>> type(True)
```

```
<class 'bool'>
```

```
>>> type(False)
```

```
<class 'bool'>
```

String:

```
>>> print('Science college')
```

```
Science college
```

```
>>> type("My college")
```

```
<class 'str'>
```

Variables

One can store integers, decimals or characters in variables.

Rules for Python variables:

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)

`a= 100` `# An integer assignment`

`b = 1040.23` `# A floating point`

`c = "John"` `# A string`

List, Tuple, Set, Dictionary

Four built-in data structures in Python:-

list, tuple, set, dictionary

- each having qualities and usage different from the other three.

List is a collection of items that is written with square brackets. It is mutable, ordered and allows duplicate members. Example: `list = [1,2,3,'A','B',7,8,[10,11]]`

Tuple is a collection of objects that is written with first brackets. It is immutable. Example: `tuple = (2, 1, 10, 4, 7)`

Set is a collection of elements that is written with curly brackets. It is unindexed and unordered. Example: `S = {x for x in 'abracadabra' if x not in 'abc'}`

Dictionary is a collection which is ordered, changeable and does not allow duplicates. It is written with curly brackets and objects are stored in key: value format. Example: `X = {1:'A', 2:'B', 3:'c'}`

print function

```
>>>type(print)
```

Output:

```
builtin_function_or_method
```

```
>>>print( 'Good morning' ) or print("Good morning")
```

Output:

```
Good morning
```

```
>>>print("Workshop", "on", "Python") or print("Workshop on Python")
```

Output:

```
Workshop on Python
```

print function

```
>>>print('Workshop', 'on', 'Python', sep='\n')
```

sep='\n' will put each word in a new line

Output:

Workshop

on

Python

```
>>>print('Workshop', 'on', 'Python', sep=', ')
```

sep=', ' will print words separated by ,

Output:

Workshop, on, Python

print function

%d is used as a placeholder for integer value.

%f is used as a placeholder for decimal value.

%s is used as a placeholder for string.

```
a = 2
```

```
b = 'tiger'
```

```
print(a, 'is an integer while', b, 'is a string.')
```

Output:

2 is an integer while tiger is a string.

Alternative way:

```
print("%d is an integer while %s is a string."%(a, b))
```

Output:

2 is an integer while tiger is a string.

print function

printing a string

```
name = "Rahul"
```

```
print("Hey " + name)
```

Output:

```
Hey Rahul
```

```
print("Roll No: " + str(34))    # "Roll No: " + 34  is incorrect
```

Output:

```
Roll No: 34
```

printing a bool True / False

```
print(True)
```

Output:

```
True
```


print function

```
int_list = [1, 2, 3, 4, 5]
```

```
print(int_list)    # printing a list
```

Output: [1, 2, 3, 4, 5]

```
my_tuple = (10, 20, 30)
```

```
print(my_tuple)    # printing a tuple
```

Output: (10, 20, 30)

```
my_dict = {"language": "Python", "field": "data science"}
```

```
print(my_dict)     # printing a dictionary
```

Output: {"language": "Python", "field": "data science"}

```
my_set = {"red", "yellow", "green", "blue"}
```

```
print(my_set)      #printing a set
```

Output: {"red", "yellow", "green", "blue"}

print function

```
str1 = 'Python code'
```

```
str2 = 'Matlab code'
```

```
print(str1)
```

```
print(str2)
```

Output: Python code

Matlab code

```
print(str1, end=' ')
```

```
print(str2)
```

Output: Python code Matlab code

```
print(str1, end=', ')
```

```
print(str2)
```

Output: Python code, Matlab code

print function

```
items = [ 1, 2, 3, 4, 5]
```

```
for item in items:
```

```
    print(item)
```

Output:

1

2

3

4

5

```
items = [ 1, 2, 3, 4, 5]
```

```
for item in items:
```

```
    print(item, end=' ')
```

Output:

1 2 3 4 5

Operators

Addition +

Multiplication *

Division /

Remainder %

Binary left shift <<

And &

Less than <

Less than or equal to <=

Check equality ==

Subtraction -

Exponentiation **

Integer division //

Binary right shift >>

Or |

Greater than >

Greater than or equal to >=

Check not equal !=

Precedence of operators

Parenthesized expression

Exponentiation

Positive, negative, bitwise not

Multiplication, float division, int division, remainder

Addition, subtraction

Bitwise left, right shifts

Bitwise and

Bitwise or

Membership and equality tests

not, <, <=, >, >=, !=, ==

Boolean (logical) not

Boolean and

Boolean or

Conditional expression

(.....)

**

+n, -n, ~n

*, /, //, %

+, -

<<, >>

&

|

in, not in, is, is

not x

and

or

if else

Precedence of Operators

Examples:

a = 20

b = 10

c = 15

d = 5

e = 2

f = (a + b) * c / d

print(f)

g = a + (b * c) / d - e

print(g)

h = a + b*c**e

print(h)

Multiple Assignment

Python allows you to assign a single value to several variables simultaneously.

```
a = b = c = 1.5
```

```
a, b, c = 1, 2, "Red"
```

Here, two integer objects with values 1 and 2 are assigned to variables a and b respectively and one string object with the value "Red" is assigned to the variable c.

Special Use of + and *

Examples:

```
x = "Python is "
```

```
y = "awesome."
```

```
z = x + y
```

```
print(z)
```

Output:

Python is awesome.

```
print('It is' + 2*'very ' + 'hot.')
```

Output:

It is very very hot.

Use of \", \n, \t

Specifying a backslash (\) in front of the quote character in a string “escapes” it and causes Python to suppress its usual special meaning. It is then interpreted simply as a literal single quote character:

```
>>> print("\'Beauty of Flower\' ")
```

```
\'Beauty of Flower\'
```

```
>>> print('Red \n Blue \n Green ')
```

```
Red
```

```
Blue
```

```
Green
```

```
>>> print("a \t b \t c \t d")
```

```
a      b      c      d
```

Comments

Single-line comments begins with a hash (#) symbol and is useful in mentioning that the whole line should be considered as a comment until the end of line.

A Multi line comment is useful when we need to comment on many lines. In python, triple double quote(“ “ “) and single quote(‘ ‘ ‘)are used for multi-line commenting.

Example:

““““ My Program to find

Average of three numbers ””””

```
a = 29      # Assigning value of a
```

```
b = 17      # Assigning value of b
```

```
c = 36      # Assigning value of c
```

```
average = ( a + b + c)/3
```

```
print(“Average value is ”, average)
```

id() function, ord() function

id() function: It is a built-in function that returns the unique identifier of an object. The identifier is an integer, which represents the memory address of the object. The id() function is commonly used to check if two variables or objects refer to the same memory location.

```
>>> a=5  
>>> id(a)  
1403804521000552
```

ord() function: It is used to convert a single unicode character into its integer representation.

```
>>> ord('A')  
65  
>>> chr(65)  
'A'
```

Control Flow Structures

1. Conditional if (if)
2. Alternative if (if else)
3. Chained Conditional if (if elif else)
4. While loop
5. For loop

Conditional if

Example:

```
a=10
```

```
if a > 9 :
```

```
    print("a is greater than 9")
```

Output:

```
a is greater than 9
```

Alternative if

Example:

```
A = int(input('Enter the marks '))  
if A >= 40:  
    print("PASS")  
else:  
    print("FAIL")
```

Output:

```
Enter the marks 65  
PASS
```

Test if the given letter is vowel or not

```
letter = 'o'
if letter == 'a' or letter == 'e' or letter == 'i' or letter == 'o'
    or letter == 'u':
    print(letter, 'is a vowel.')
else:
    print(letter, 'is not a vowel.')
```

Output:

o is a vowel.

Program to find the greatest of three different numbers

```
a = int(input('Enter 1st no'))
b = int(input('Enter 2nd no'))
c = int(input('Enter 3rd no'))

if a > b:
    if a > c:
        print('The greatest no is ', a)
    else:
        print('The greatest no is ', c)
else:
    if b > c:
        print('The greatest no is ', b)
    else:
        print('The greatest no is ', c)
```

Output:

Enter 1st no 12

Enter 2nd no 31

Enter 3rd no 9

The greatest no is 31

Chained conditional if

Program to guess the vegetable

```
color = "green"
if color == "red":
    print('It is a tomato.')
elif color == "purple":
    print('It is a brinjal.')
elif color == "green":
    print('It is a papaya. ')
else:
    print('There is no such vegetable.')
```

Output:

It is a papaya.

Program to find out the greatest of four different numbers

```
a=int(input('Enter 1st no '))
b=int(input('Enter 2nd no '))
c=int(input('Enter 3rd no '))
d=int(input('Enter 4th no '))
if (a>b and a>c and a>d):
    print('The greatest no is ', a)
elif (b>c and b>d):
    print('The greatest no is ', b)
elif (c>d):
    print('The greatest no is ', c)
elif d>c :
    print('The greatest no is ', d)
else:
    print('At least two values are equal')
```

Output:

Enter 1st no 23

Enter 2nd no 10

Enter 3rd no 34

Enter 4th no 7

The greatest no is 34

Program to find out Grade

```
marks = int(input('Enter total marks '))  
total = 500    # Total marks  
percentage=(marks/total)*100  
if percentage >= 80:  
    print('Grade O')  
elif percentage >=70:  
    print('Grade A')  
elif percentage >=60:  
    print('Grade B')  
elif percentage >=40:  
    print('Grade C')  
else:  
    print('Fail')
```

Output:

Enter total marks 312

Grade B

While loop

Python program to find first ten Fibonacci numbers

```
a=1
```

```
print(a)
```

```
b=1
```

```
print(b)
```

```
i=3
```

```
while i<= 10:
```

```
    c=a+b
```

```
    print(c)
```

```
    a=b
```

```
    b=c
```

```
    i=i+1
```

For loop

Program to find the sum of squares of first n natural numbers

```
n = int(input('Enter the last number '))
```

```
sum = 0
```

```
for i in range(1, n+1):
```

```
    sum = sum + i*i
```

```
print('The sum is ', sum)
```

Output:

Enter the last number 8

The sum is 204

For loop

Program to find the sum of a given set of numbers

```
numbers = [11, 17, 24, 65, 32, 69]
```

```
sum = 0
```

```
for no in numbers:
```

```
    sum = sum + no
```

```
print('The sum is ', sum)
```

Output:

The sum is 218

Program to print 1, 22, 333, 444, in triangular form

```
n = int(input('Enter the number of rows '))  
for i in range(1, n+1):  
    for j in range(1, i+1):  
        print(i, end="")  
    print()
```

Output:

Enter the number of rows 5

1

22

333

4444

55555

Program to print opposite right triangle

```
n = int(input('Enter the number of rows '))  
for i in range(n, 0, -1):  
    for j in range(1, i+1):  
        print('*', end="")  
    print()
```

Output:

**

*

Program to print opposite star pattern

```
n = int(input('Enter the number of rows '))  
for i in range(0, n):  
    for j in range(0, n-i):  
        print(' ', end="")  
    for k in range(0, i+1):  
        print('*', end="")  
    print("")
```

Output:

```
      *  
     **  
    ***  
   ****  
  *****
```

Program to print A, AB, ABC, ABCD,

```
ch = str(input('Enter a character '))
```

```
val=ord(ch)
```

```
for i in range(65, val+1):
```

```
    for j in range(65, i+1):
```

```
        print(chr(j), end="")
```

```
print()
```

Output:

A

AB

ABC

ABCD

ABCDE

ABCDEF

Program to test Palindrome numbers

```
n=int(input('Enter an integer '))

x=n

r=0

while n>0:

    d=n%10

    r=r*10+d

    n=n//10

    if x==r:

        print(x,' is Palindrome number')

    else:

        print(x, ' is not Palindrome number')
```

Program to print Pascal Triangle

```
n=int(input('Enter number of rows '))
```

```
for i in range(0, n):
```

```
    for j in range(0, n-i-1):
```

```
        print(end=' ')
```

```
    for j in range(0, i+1):
```

```
        print('*', end=' ')
```

```
    print()
```

Output:

Enter number of rows 6

```
      *
     * *
    * * *
   * * * *
  * * * * *
 * * * * * *
```

Break and Continue

In Python, break and continue statements can alter the flow of a normal loop.

Searching for a given number

```
numbers = [11, 9, 88, 10, 90, 3, 19]
```

```
for num in numbers:
```

```
    if(num==88):
```

```
        print("The number 88 is found")
```

```
        break
```

Output:

The number 88 is found

Break and Continue

program to display only odd numbers

```
for num in [20, 11, 9, 66, 4, 89, 44]:
```

```
    # Skipping the iteration when number is even
```

```
    if num%2 == 0:
```

```
        continue
```

```
        # This statement will be skipped for all even numbers
```

```
    else:
```

```
        print(num)
```

File

A file is some information or data which stays in the computer storage devices.

Files are of two types:

- text files
- binary files.

Text files:

We can create the text files by using the following syntax:

Variable name = open ("file.txt", file mode)

Example:

```
f= open ("hello.txt","w+")
```

File modes

Mode	Description
'r'	This is the default mode. It Opens file for reading.
'w'	This Mode Opens file for writing. If file does not exist, it creates a new file. If file exists it truncates the file.
'x'	Creates a new file. If file already exists, the operation fails.
'a'	Open file in append mode. If file does not exist, it creates a new file.
't'	This is the default mode. It opens in text mode.
'b'	This opens in binary mode.
'+'	This will open a file for reading and writing (updating) .

Creating output file

```
file = open('output.txt', 'a+')
```

```
items = ['mango', 'orange', 'banana', 'apple']
```

```
for item in items:
```

```
    print(item, file = file)
```

```
file.close()
```

Write a python program to open and read a file

```
a=open("one.txt", "r")  
content = a.read()  
print(content)  
a.close( )
```

Write a python program to open and write "hello world" into a file.

```
f=open("file.txt","a")  
f.write("hello world")  
f.close( )
```

Python program to write the content “Hi python programming” for the existing file.

```
f=open("MyFile.txt",'w')  
f.write("Hi python programming")  
f.close()
```

Write a python program to open and write the content to file and read it.

```
f=open("abc.txt","w+")  
f.write("Python Programming")  
print(f.read())  
f.close()
```

References

- [1] Kenneth A Lambert, Fundamentals of Python: First programs, 2nd edition – Cengage Learning India, 2019.
- [2] Saha Amit, Doing Math with Python - No starch press, San Francisco, 2015.
- [3] E. Balgurusamy, Problem solving and Python programming- Tata McGraw Hill, 2017.
- [4] Bill Lubanovic, Introducing Python, Shroff Publishers & Distributors Pvt. Ltd., 2nd Edition, 2020.

Understanding a Python list and useful list functions

Creating a list of variables

```
Park_Avenue = 545  
Hill_View = 47  
North_Hill = 672  
area = [Park_Avenue, Hill_View, North_Hill]  
print(area)
```

```
#OUTPUT  
[545,47,672]
```

Subsets of lists or nested lists

Moreover, lists in Python can also consist of sub-lists as elements, and such lists are referred to as nested lists.

```
to_do = ['laundry', 'cooking', 'report writing', ['milk', 'eggs', 'bread', 'butter'], 'gardening']
```

Understanding a Python list and useful list functions

Determining the length of a list by len function

```
cake = ['flour', 'eggs', 'sugar', 'butter', 'baking powder', 'baking soda', 'vanilla essence']  
print(len(cake))
```

```
#OUTPUT  
7
```

Accessing a specific element by its index

```
cake = ['flour', 'eggs', 'sugar', 'butter', 'baking powder', 'baking soda', 'vanilla essence']  
print(cake[2])
```

```
#OUTPUT  
sugar
```

```
to_do = ['laundry', 'cooking', 'report writing', ['milk', 'eggs', 'bread', 'butter'], 'gardening']  
print(to_do[3][1])
```

```
#OUTPUT  
eggs
```

Slicing a Python list by its index

```
cake = ['flour', 'eggs', 'sugar', 'butter', 'baking powder', 'baking soda', 'vanilla essence']  
print(cake[1:4])
```

```
#OUTPUT  
['eggs', 'sugar', 'butter', 'baking powder']
```

Understanding a Python list and useful list functions

Counting the occurrence of an element in a list

```
squares = [4, 9, 16, 4, 4, 49, 81, 4]
print(squares.count(4))
```

#OUTPUT

2

Sorting list elements

```
a = [9, 3, 15, 30, 27, 12]
b = a.sort()
print(a,b)
```

#OUTPUT

[3, 9, 12, 15, 27, 30] None

```
a = [9, 3, 15, 30, 27, 12]
b = sorted(a)
print(a,b)
```

#OUTPUT

[3, 9, 12, 15, 27, 30] [3, 9, 12, 15, 27, 30]

`list.reverse( )` or `reverse(list)` the list can be arranged in descending order.

```
a = [9, 3, 15, 30, 27, 12]
a.sort(reverse=True)
print(a)
```

#OUTPUT

[30, 27, 15, 12, 9, 3]

```
a = [9, 3, 15, 30, 27, 12]
a.reverse()
print(a)
```

#OUTPUT

[30, 27, 15, 12, 9, 3]

Understanding a Python list and useful list functions

Manipulating list elements by replacing a specific item

```
cake = ['flour', 'eggs', 'sugar', 'butter', 'baking powder', 'baking soda', 'vanilla essence']  
#Determining index of sugar  
i = cake.index('sugar')  
#Replacing sugar with Brown sugar  
cake[i] = 'Brown sugar'  
print(cake)
```

```
#OUTPUT  
['flour', 'eggs', 'Brown sugar', 'butter', 'baking powder', 'baking soda', 'vanilla essence']
```

Manipulating list elements by adding new items with append()

```
cake = ['flour', 'eggs', 'sugar', 'butter', 'baking powder', 'baking soda', 'vanilla essence']  
cake.append('cocoa powder')  
print(cake)
```

```
#OUTPUT  
['flour', 'eggs', 'sugar', 'butter', 'baking powder', 'baking soda', 'vanilla essence', 'cocoa powder']
```

```
cake = ['flour', 'eggs', 'sugar', 'butter', 'baking powder', 'baking soda', 'vanilla essence']  
cake.append(['cocoa powder', 'apple juice'])  
print(cake)
```

```
#OUTPUT  
['flour', 'eggs', 'sugar', 'butter', 'baking powder', 'baking soda', 'vanilla essence', ['cocoa powder', 'apple juice']]
```


Understanding a Python list and useful list functions

Inserting an item to a specific position in a list

New items can be added to an existing list at specific positions with `list.insert()`.

```
cake = ['flour', 'eggs', 'sugar', 'butter', 'baking powder', 'baking soda', 'vanilla essence']
cake.insert(1, 'cocoa powder')
print(cake)
```

#OUTPUT

```
['flour', 'cocoa powder', 'eggs', 'sugar', 'butter', 'baking powder', 'baking soda', 'vanilla essence']
```

Merging two different lists with extend()

`list.extend(newList)` combines 'newList' to 'list'.

```
a = [ 9, 3, 15, 30]
b = [12, 24, 6, 18]
a.extend(b)
print(a)
```

#OUTPUT

```
[9, 3, 15, 30, 12, 24, 6, 18]
```

Understanding a Python list and set

Concatenating lists in Python

```
alpha = ['a','b','c','d']  
num = [1,2,3,4,5]  
print(alpha + num)
```

#OUTPUT

```
['a', 'b', 'c', 'd', 1, 2, 3, 4, 5]
```

Changing lists to sets in Python

sets do not store multiple occurrences of values

```
a = ['a','b','c','c']  
s = set(a)  
print(s)
```

#OUTPUT

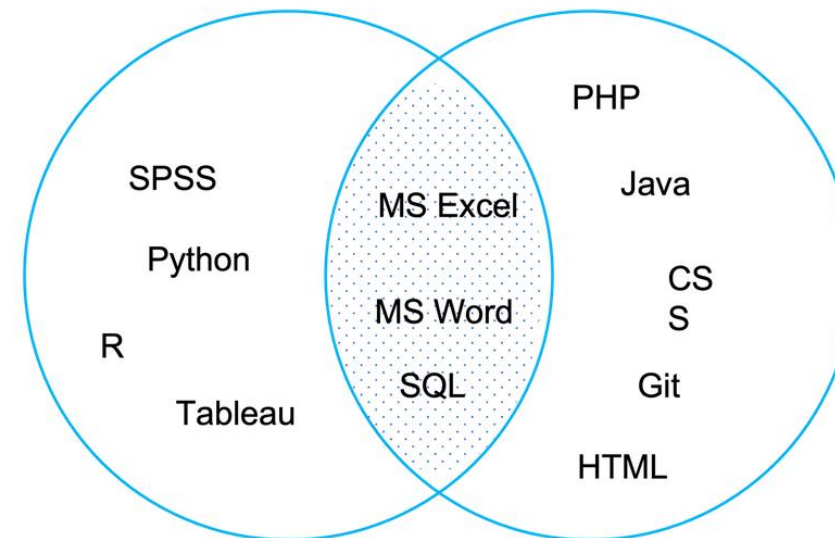
```
{'b', 'c', 'a'}
```

The intersection of two sets in Python

```
a = ['SPSS','Python','R','Tableau','MS Excel','MS Word','SQL']  
b = ['PHP','Java','CSS','Git','HTML','MS Excel','MS Word','SQL']  
c = set(a) & set(b)  
print(c)
```

#OUTPUT

```
{'MS Word', 'MS Excel', 'SQL'}
```



The intersection of two different skill sets

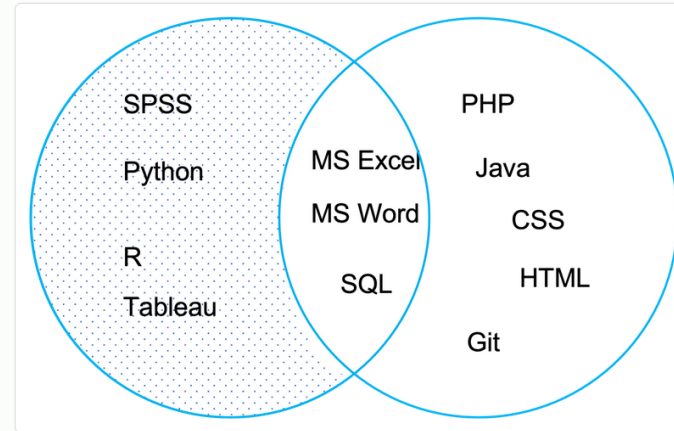
Understanding a Python list and set

Difference between the two sets

```
a = ['SPSS','Python','R','Tableau','MS Excel','MS Word','SQL']
b = ['PHP','Java','CSS','Git','HTML','MS Excel','MS Word','SQL']
c = set(a) - set(b)
print(c)
```

#OUTPUT

```
{'R', 'SPSS', 'Tableau', 'Python'}
```



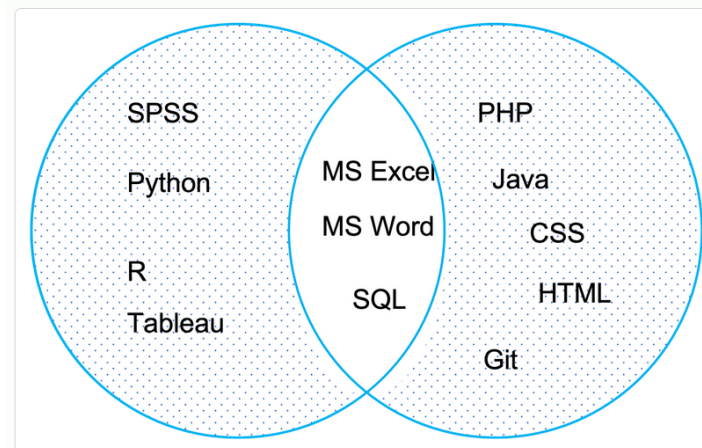
Different between sets

Symmetric difference between the two sets

```
a = ['SPSS','Python','R','Tableau','MS Excel','MS Word','SQL']
b = ['PHP','Java','CSS','Git','HTML','MS Excel','MS Word','SQL']
c = set(a) ^ set(b)
print(c)
```

#OUTPUT

```
{'R', 'SPSS', 'Java', 'CSS', 'HTML', 'Python', 'PHP', 'Git', 'Tableau'}
```



The symmetric difference in sets

Pandas and Matplotlib for Qualitative analysis

Pandas and Matplotlib are Python libraries designed to make working with structured data fast, flexible, and intuitive.

Pandas provides tools to clean, process, and analyze datasets efficiently.

Matplotlib is useful for creating static, interactive, and publication-quality visualizations.

Series and DataFrame in Pandas

Dictionary

```
fruitPrice = ['apple' : 10, 'banana', : 20, 'cherry' : 30]

print(fruitPrice["cherry"])
```

30

Pandas Series

```
import pandas as pd

fruitPanda = pd.Series([10, 20, 30], index=[ 'apple', 'banana', 'cherry'])

print(fruitPanda ['apple'])
```

10

The key distinctions of Pandas series are:

- It supports more data analysis operations.
- It maintains type consistency.
- It has built-in statistical methods.
- It can perform vectorized operations.

Pandas and Matplotlib for Qualitative analysis

A DataFrame is a two-dimensional labeled data structure in Pandas. It is the most used structure and is also analogous to a table in a database, Excel sheet, or spreadsheet.

A DataFrame is made up of multiple Series.

DataFrame has both row and column labels for organized data representation.

The columns in a DataFrame can have different data types.

```
import pandas as pd

data = {
    "Name": ["Amar", "Baba", "Ramesh"],
    "Age": [25, 30, 35],
    "Gender": ["Male", "Male", "Male"]
}

df = pd.DataFrame (data)

print(df)
```

	Name	Age	Gender
0	Amar	25	Male
1	Baba	30	Male
2	Ramesh	35	Male

```
import pandas as pd

data = {
    "Name": ["Amar", "Baba", "Ramesh"],
    "Age": [25, 30, 35],
    "Gender": ["Male", "Male", "Male"]
}

df = pd.DataFrame(data)

print(df["Name"]) # Access by column

print(df.iloc[1]) # Access row by integer index

print(df[["Name", "Age"]]) # Select specific columns
```

#By column

```
0 Amar
1 Baba
2 Ramesh
Name: Name, dtype: object
```

#By integer index

```
Name Baba
Age 30
Gender Male
Name: 1, dtype : object
```

#By specific columns

```
   Name  Age
0  Amar   25
1  Baba   30
2  Ramesh  35
```

Pandas and Matplotlib for Qualitative analysis

Importing datasets as a DataFrame for analysis

Pandas support importing a variety of file formats for analysis. Below are the commonly used ones:

- **CSV (.csv):** Comma-separated values (CSV) is a text file.
 - `df = pd.read_csv("data.csv")`
- **Excel files (.xls, .xlsx):** Requires the library [openpyxl](#) or [xlrd](#) for reading.
 - `df = pd.read_excel("data.xlsx", sheet_name = "Sheet1")`
- **JSON (.json):** Semi-structured data, often used in web applications.
 - `df = pd.read_json("data.json")`
- **HTML (.html):** Extracting tabular data from web pages.
 - `pd.read_html("https://example.com/table")`
- **Pickle Files (.pkl):** Serialized Python objects.
 - `df = pd.read_pickle("data.pkl")`
- **Clipboard:** Copy and paste.
 - `df = pd.read_clipboard()`

Pandas and Matplotlib for Qualitative analysis

Essential Pandas methods to learn before working on a dataset

Mastering a set of fundamental Pandas methods is crucial to working confidently on datasets. These methods are useful in data exploration, data cleaning, data manipulation, and analytics.

Data exploration methods that help to understand the structure and content of the dataset are:

- View the first or last rows of a dataset using `df.head(n)` for the first and `df.tail(n)` for the last `n` number of rows.
- `df.info()` displays a summary of the dataset, including column names, data types, and non-null values.
- `df.shape` provides a summary of the number of rows and columns in the dataset.
- `df.columns` and `df.index` lists column names and row labels.
- To generate summary statistics such as mean, standard deviation, and percentiles for numeric columns use `df.describe()`.

Data cleaning methods that help to make the dataset ready for analysis by handling missing values, duplicates, and inconsistencies:

- Check missing values in the dataset with `df.isnull()` or `df.notnull()`. Both methods return Boolean values (true/false) commonly used in conditional statements.
- To handle missing values in a dataset use `df.fillna(n)`, where 'n' is the value to be inserted. Use `df.dropna()` to drop the rows with missing values.
- Identify duplicates in a dataset with `df.duplicated()` and remove duplicated rows with `df.drop_duplicates()`.
- Rename columns or indexes as `df.rename(columns = {"old_name": "new_name"})`.
- To change the type of a column use `df["column_name"] = df["column_name"].astype(float)`.

Pandas and Matplotlib for Qualitative analysis

Data manipulation methods that help to filter, sort, and reshape the dataset for analysis:

- To select specific rows and columns based on labels use `df.loc[from: to, "column_name"]`. For rows 0 – 5 replace `from:to` with `0:5`. To select by positions use `df.iloc[FromRows:ToRows, FromCols:ToCols]`. For rows 0 – 5 replace `FromRows:ToRows` with `0:5` & for columns 1 – 3 replace `FromCols:ToCols` with `1:3`.
- To sort data by one or more columns use `df.sort_values(by="column_name", ascending=True)`. Set ascending to false for descending order.
- Remove specific columns `df.drop("column_name", axis=1)` and rows with `df.drop(0, axis=0)`.
- To apply a custom function containing calculations or manipulation to a column or row use `df["column_name"].apply(lambda x: x * 2)`.
- To group data by categorical values and apply aggregate functions use `df.groupby("category")["value_column"].mean()`.
- Combine or merge different datasets as `pd.merge(df1, df2, on="common_column")`
- Convert data into a DateTime format as `pd.to_datetime(arg, format=None, errors='raise', exact=True, unit=None)`
 - **arg**: The input data to convert (e.g., a column, list, or Series).
 - **format**: A string representing the format of the input data (e.g., `%Y-%m-%d`). Optional.
 - **errors**: Specifies how to handle invalid parsing (`'raise'`, `'coerce'`, `'ignore'`).
 - **unit**: Specifies the unit of the timestamp (e.g., `s` for seconds).

Pandas and Matplotlib for Qualitative analysis

Basic analytics methods that help to perform descriptive and exploratory analysis on a dataset.

- Count occurrences of unique values in a column with `df["column_name"].value_counts()`
- Calculate the mean, median, and standard deviation for numeric data as `df["column_name"].mean()`, `df["column_name"].median()`, `df["column_name"].std()`
- To calculate the correlation matrix between columns use `df.corr()`. To calculate the correlation between specific columns use `df['column1'].corr(df['column2'])`.
- To create pivot tables for summarizing data use `df.pivot_table(index="column1", values="column2", aggfunc="mean")`.
- Perform multiple aggregate functions on grouped data using `df.groupby("category").agg({"value_column": ["mean", "sum"]})`.

Pandas and Matplotlib for Qualitative analysis

Using Matplotlib to visualize data

Representing data graphically using charts, graphs, and plots helps to make the data more accessible, interpretable, and actionable, especially for complex datasets. In quantitative studies, datasets involve multivariate relationships, trends, and patterns, where visualization plays a critical role in communication.

While tools like Excel, [SPSS](#), and [STATA](#) are powerful, Python's Matplotlib offers unique advantages such as:

- **Customization and Flexibility:** Control every aspect of the chart, from colour schemes and markers to labels and legends.
- **Scalability for Large Datasets:** Excel and [SPSS](#) struggle with very large datasets and require manual interventions for repetitive tasks. Matplotlib can handle large datasets programmatically and can also automate repetitive visualizations.
- **Advanced Visualizations:** Matplotlib supports advanced plots like heat maps, 3D plots, and subplots for multi-dimensional data.
- **Cost-Effectiveness and Accessibility:** Matplotlib is Open-source and free to use, making it accessible to all.

While traditional tools like Excel, [SPSS](#), and [STATA](#) are reliable and familiar, Matplotlib complements them by offering unmatched flexibility, scalability, and integration capabilities. Matplotlib works seamlessly with Pandas, making it ideal for structured dataset visualization. There are several components of Matplotlib:

- **Figure:** The overall window or page where your plot appears.
- **Axes:** The area where the data is plotted (e.g., within the X and Y axes).
- **Plot Elements:** Titles, labels, legends, and gridlines that enhance readability.

Pandas and Matplotlib for Qualitative analysis

To create a plot, from a Pandas data frame import the Pandas and Pyplot module. They both provide functions for preparing the dataset and plotting the charts.

```
import matplotlib.pyplot as plt
import pandas as pd

# Sample data

data = {'Year': [2018, 2019, 2020, 2021],
        'Literacy_Rate': [75.2, 76.3, 77.5, 78.6]}

df = pd.DataFrame(data)

# Create a line plot

plt.plot(df['Year'], df['Literacy_Rate'], marker='o', color='blue', linestyle='--')

# Add title and labels

plt.title("Literacy Rate Over Time")

plt.xlabel("Year")

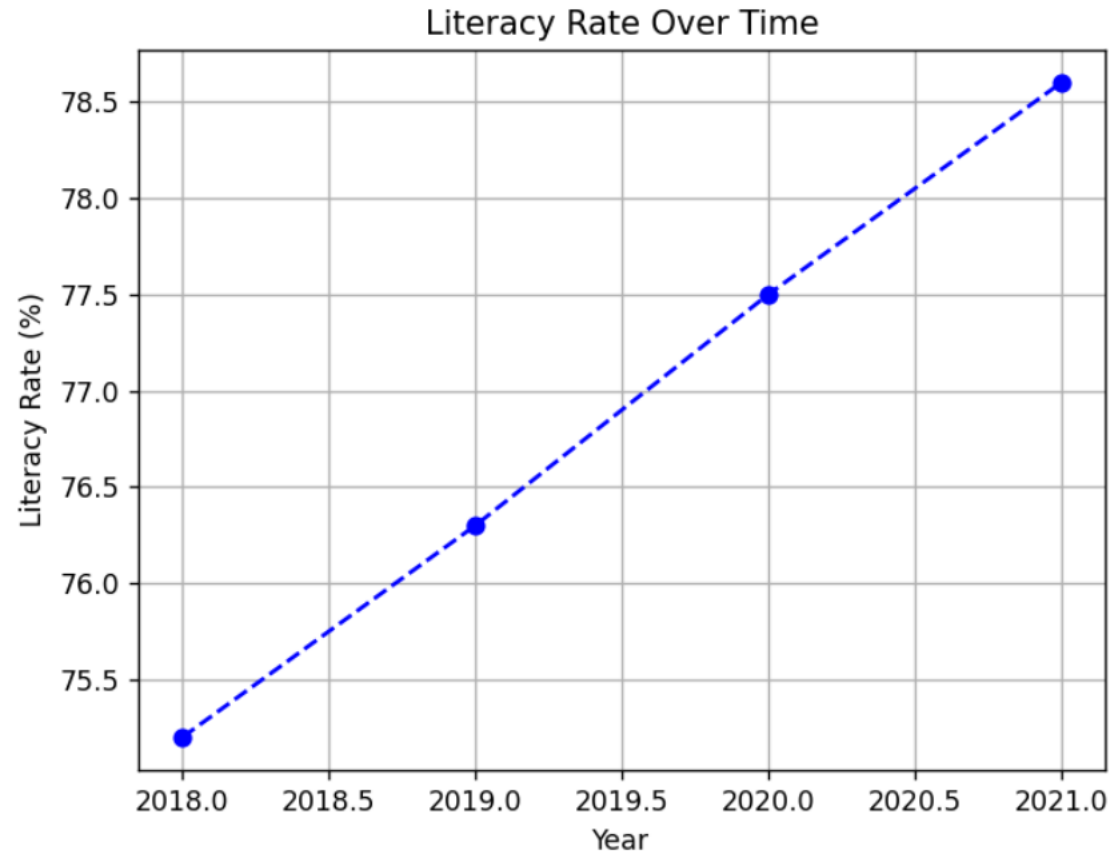
plt.ylabel("Literacy Rate (%)")

# Show the plot

plt.grid(True) # Show gridlines

plt.show()
```

Pandas and Matplotlib for Qualitative analysis



Pandas and Matplotlib for Qualitative analysis

Key Matplotlib methods and their arguments

`plot(x, y, color, marker, linestyle)`

- x: Data for the x-axis.
- y: Data for the y-axis.
- color: Line colors like 'r' for red, 'b' for blue, etc.
- marker: Marker style like 'o' for circles, 's' for squares.
- line style: Style of the plot line like '-', '--', ':'.

```
plt.plot([1, 2, 3], [4, 5, 6], color='g', marker='o', linestyle='--')
```

`bar(x, height, color, width)`

- x: Categories for the x-axis.
- height: Heights of the bars (values).
- color: Fill color of the bars.
- width: Width of the bars (default is 0.8).

```
plt.bar(['A', 'B', 'C'], [10, 20, 15], color='orange', width=0.5)
```

`scatter(x, y, color, alpha, s)`

- x, y: Data points for x and y axes.
- color: Color of the points like 'red' or '#ff5733'.
- alpha: Transparency of the points like value s between 0 and 1.
- s: Size of the markers as integers.

```
plt.scatter([1, 2, 3], [4, 5, 6], color='blue', alpha=0.7, s=100)
```

`hist(x, bins, color, edgecolor)`

- x: Data to create a histogram
- bins: Number of bins (default is 10)
- color: Fill color of the bars
- edgecolor: Color of the edges of the bars

```
plt.hist([1, 2, 2, 3, 3, 3, 4, 4, 5], bins=5, color='purple', edgecolor='black')
```

Pandas and Matplotlib for Qualitative analysis

`boxplot(x, notch, vert)`

- x: List or array of data to be plotted
- notch: Whether to draw notches in the box (default is False)
- vert: Orientation of the plot (True for vertical, False for horizontal)

```
plt.boxplot ([7, 8, 9, 10, 12, 15], notch=True, vert=True)
```

`title(label, fontsize, loc)`

- label: Text for the title
- fontsize: Font size of the title
- loc: Location of the title ('left', 'center', 'right')

```
plt.title ("My Plot Title", fontsize =14, loc=' center ')
```

`pie(x, labels, autopct, startangle)`

- x: Sizes of each wedge
- labels: Labels for each wedge
- autopct: Format for displaying percentages (e.g., '%1.1f%%')
- startangle: Angle to start the pie chart (default is 0)

```
plt.pie([20, 30, 50], labels=['A', 'B', 'C'], autopct = '%1.1f%%', startangle =90)
```

`xlabel(label, fontsize)` and `ylabel(label, fontsize)`

- label: Text for the axis labels.
- fontsize: Font size of the labels.

```
plt.xlabel ("X-Axis Label", fontsize =12)
```

```
plt.ylabel ("Y-Axis Label", fontsize =12)
```

Pandas and Matplotlib for Qualitative analysis

`grid(visible, color, linestyle, linewidth)`

- visible: Whether gridlines are visible (True or False)
- color: Color of the gridlines
- linestyle: Style of the gridlines ('-', ':')
- linewidth: Thickness of the gridlines

```
plt.grid (visible=True, color =' gray ', linestyle ='--', linewidth=0.5)
```

- `show()` : Create and display the chart.

```
plt.show()
```

`legend(loc, fontsize)`

- loc: Location of the legend ('upper right', 'lower left', etc.)
- fontsize: Font size of the legend text

```
plt.legend (loc='upper left', fontsize =10)
```

`savefig(fname, dpi, bbox_inches)`

- fname: File name or path (e.g., 'plot.png' or 'folder/plot.pdf')
- dpi: Dots per inch for resolution
- bbox_inches: Bounding box for the plot ('tight' for no extra white space)

```
plt.savefig ("my_plot.png", dpi=300, bbox_inches ='tight')
```

Exercise: Analyse e-commerce transactions & plot charts

- Use Pandas to load the dataset and inspect its structure.
- Identify key columns in the dataset.
- Visualize the distribution of orders in different categories using the histogram.
- Create a bar chart showing the top 10 customers by transaction count. Highlight customers with high transaction frequencies.
- Plot a line graph to analyze how order amounts change over time. Detect unusual spikes in order amounts on specific dates.

	A	B	C	D	E	F	G	H
1	User_ID	Product_ID	Category	Price (Rs.)	Discount (%)	Final_Price(Rs.)	Payment_Method	Purchase_Date
2	337c166f	f414122f-e	Sports	36.53	15	31.05	Net Banking	11/12/2024
3	d38a19bf	fde50f9c-5	Clothing	232.79	20	186.23	Net Banking	2/9/2024
4	d7f5f0b0	0d96fc90-3	Sports	317.02	25	237.76	Credit Card	9/1/2024
5	395d4994	964fc44b-d	Toys	173.19	25	129.89	UPI	4/1/2024
6	a83c145c	d70e2fc6-e	Beauty	244.8	20	195.84	Net Banking	9/27/2024
7	3fdcdae8	0816ee12-5	Books	241.86	50	120.93	UPI	8/8/2024

Dataset

THANK YOU